

Çok yerinde bir soru. "Nasıl çalışıyor" değil "neden doğru çalışıyor" — bunu matematiksel olarak kanıtlayalım.

Bu Mantık Neden Doğru?

Bir görsel hareket motorunun **3 şartı** sağlaması gerekir:

Şart 1: Sıçrama Olmamalı (Süreklilik)

Paket geldiği anda görsel konum **hiç değişmemeli**. Bunu test edelim:

Paket gelmeden 1ms ÖNCE:

Görsel = targetPos_eski = 10.0m (ekstrapolasyonla buraya geldik)

Paket geldiği AN (t=0):

Hata yakalanır: E = 10.0 - 9.8 = +0.2

targetPos_yeni = 9.8 (yeni baz)

Katsayı = $e^{(-decay \times 0)} = e^0 = 1.0$

Görsel = 9.8 + 0.2 × 1.0 = 10.0m ← DEĞİŞMEDİ ☒

Bu matematiksel bir **garanti**: $baz + (görsel - baz) \times 1 = görsel$. Her zaman, her koşulda. Hata ne kadar büyük olursa olsun sıçrama olmaz.

Şart 2: Zamanla Doğruya Yakınsamalı (Convergence)

Görsel konum, sonunda ekstrapolasyonun (fizik tahmininin) üzerine oturmalı:

t → büyüdükçe:

Katsayı = $e^{(-decay \times t)} \rightarrow 0$

Görsel = targetPos + 0.2 × 0 = targetPos ← Artık saf fizik tahmini ☒

Üstel fonksiyon asla tam 0 olmaz ama pratikte 200ms sonra $e^{(-3)} = 0.05 \rightarrow$ hatanın %95'i erimiş olur. Bir sonraki paket gelene kadar zaten kaybolmuştur.

Şart 3: Uçak Yavaşlamamalı veya Geri Gitmemeli

Alternatif yöntemlerle karşılaştıralım:

Yöntem A — Lerp (Doğrusal Enterpolasyon):

Görsel = lerp(eski_konum, yeni_konum, t/süre)

Sorun: t/süre boyunca uçak ESKİ konumdan YENİ konuma doğru hareket eder.

Bu sürede uçak YAVAŞLAR veya ters yöne gidebilir.

Çünkü hedef sabit bir nokta, uçak o noktayı "kovalıyor".

Yöntem B — Snap (Işınlanma):

Görsel = yeni_konum (anında)

Sorun: Her pakette 5-50cm'lik sıçrama. 5Hz'de bu görünür titreme yapar.

Yöntem C — Error Blending (Bu sistemdeki):

Görsel = ilerleyen_hedef + eriyen_hata

Ekstrapolasyon TAM HIZLA ilerler (fizik tahmini hiç yavaşlamaz).

Hata sadece bir "OFFSET" olarak eklenir ve zamanla kaybolur.

Uçak hiçbir zaman yavaşlamaz veya geri gitmez.

Bunu şöyle düşün:

LERP:  —————> DURUR, hedefe doğru kayar

↓

 —————> Tekrar hızlanır

ERROR BLENDING:

 —————> Hiç durmaz!

↑ sadece yolu biraz "kaydırılmış" başlar,

sonra gerçek yola yumuşakça oturur

Neden Üstel Azalma (Exponential Decay)?

Doğrusal azalma ($1.0 - t/\text{süre}$) yerine $e^{(-kt)}$ kullanılması tesadüf değil:

Doğrusal: $\text{Hata} = E \times (1 - t/T)$

$t=T$ anında hata TAM SIFIR olur $\rightarrow$ keskin bir "köşe" oluşur

$t>T$ olursa NEGATİFE geçer (zıplama!)

Üstel: $\text{Hata} = E \times e^{(-kt)}$

Asla sıfırın altına düşmez

Asla keskin köşe yapmaz

Ne kadar büyük hata olursa o kadar hızlı erir (kendi kendini ayarlar)

Tek Cümleyle

"Paket geldiğinde sıçramayı engelle ($\text{hata} \times 1 = \text{tam telafi}$), sonra bu telafi katkısını üstel olarak sıfıra eritip uçağın fizik tahminine yumuşakça oturmasını sağla, bu sırada ekstrapolasyon tam hızla ilerlemeye devam etsin."

Bu yaklaşım ağ oyunlarında (Overwatch, Rocket League, vb.) ve askeri simülasyonlarda (DIS/HLA protokolleri) standart yöntemdir. Dead Reckoning + Exponential Error Correction olarak bilinir.

2:33 PM